<<interface>>
BombConstructor
<<enumeration>>
FireType
BomberManAliveState
+ BomberManAliveState(BomberMan, MovingImageManager, AbstractState)
- imageManager: MovingImageManager
- bomberMan: BomberMan
+ notifyExplosion(Explosion): boolean
+ updateInModel(): void
# die(): void
+ notifyCaughtByEnemy(): void
WallAliveState
+ WallAliveState(Wall, AbstractState, ImageManager):
- wall: Wall
- imageManager: ImageManager
+ updateInModel(): void
+ getDrawing(): Drawing
AliveState
# AliveState(StateFul, AbstractState):
- deathState: AbstractState
# die(): void
EnemyAliveState
+ EnemyAliveState(Enemy, MovingImageManager, AbstractState):
- imageManager: MovingImageManager
- enemy: Enemy
+ updateInModel(): void
+ getDrawing(): Drawing
ImageManager
+ ImageManager(List<AbstractImage>, int):
- updateInterval: int
- DEFAULT_UPDATE_INTERVAL: int
- timeToUpdate: int
- idxInImages: int
- hasDoneOneCycle: boolean
- images: List<AbstractImage>
+ getImage(): AbstractImage
+ setImages(List<AbstractImage>): void
+ setIdxInImages(int): void
+ setUpdateInterval(int): void
+ getCurrentImage(): AbstractImage
+ hasChanged(): boolean
MovingImageManager
# MovingImageManager(List<AbstractImage>, int):
- previousMoveDirection: Maybe<Direction>
# getImageSet(Direction): List<AbstractImage>
+ getImage(Maybe<Direction>): AbstractImage
LeftRightMovingImageManager
+ LeftRightMovingImageManager(List<AbstractImage>, List<AbstractIma
- left: List<AbstractImage>
- right: List<AbstractImage>
# getImageSet(Direction): List<AbstractImage>
FullMovementMovingImageManager
+ FullMovementMovingImageManager(List<AbstractImage>, List<Abs
- right: List<AbstractImage>
- left: List<AbstractImage>
- up: List<AbstractImage>
- down: List<AbstractImage>
Bomb
# Bomb(Vector2D, int, BomberMan, ImageManager, GameGrid):
- BOMB_DETONATION_SOUND: String
- bombRange: int
- imageManager: ImageManager
- bomberman: BomberMan
+ notifyExplosion(Explosion): boolean
+ makeExplode(): void
# updateWithExplosion(): void
+ toString(): String
+ getDrawing(): Drawing
ControlledBomb
+ ControlledBomb(Vector2D, int, BomberMan, ImageManager, GameGrid):
- doDetonation: boolean
+ updateInModel(): void
+ makeExplode(): void
AutoBomb
+ AutoBomb(Vector2D, int, BomberMan, ImageManager, GameGrid):
- BOMB_EXPLOSION_INTERVAL: int
- timeToExplosion: int
+ updateInModel(): void
+ toString(): String
GameCharacterCapabilities
+ GameCharacterCapabilities(boolean, double):
- goesThroughWallsFlag: boolean
- speed: double
+ goesThroughWalls(): boolean
+ setSpeed(double): void
+ toString(): String
+ getSpeed(): double
+ setGoesThroughWalls(boolean): void
+ getSpeedInstance(): double
Powerup
+ Powerup(Vector2D, GameGrid, AbstractImage, Enemy, BomberManCap
- POWERUP_TAKEN_MUSIC: String
- deltaCapabilities: BomberManCapabilities
+ notifyExplosion(Explosion): boolean
PickableCell
+ PickableCell(Vector2D, GameGrid, AbstractImage, Supplier<Enemy>):
- enemySupplier: Supplier<Enemy>
+ updateInModel(): void
+ notifyExplosion(Explosion): boolean
+ allowsBombs(): boolean
AbstractState
# AbstractState(StateFul, boolean):
- isDisappearing: boolean
- stateFul: StateFul
+ notifyExplosion(Explosion): boolean
+ updateInModel(): void
+ isDisappearing(): boolean
# getStateful(): StateFul
GameCharacter
# GameCharacter(Vector2D, GameCharacterCapabilities, GameGrid):
- stateIsNothingError: Supplier<JBomberManException>
- capabilities: GameCharacterCapabilities
- state: Maybe<AbstractState>
+ setState(AbstractState): void
+ updateInModel(): void
+ setCapabilities(GameCharacterCapabilities): void
+ getDrawing(): Drawing
+ notifyExplosion(Explosion): boolean
+ move(): void
+ isDisappearing(): boolean
+ getCapabilities(): GameCharacterCapabilities
Cell
# Cell(Vector2D, AbstractImage, GameGrid):
- img: AbstractImage
+ getDrawing(): Drawing
Block
+ Block(Vector2D, GameGrid):
- img: String
+ notifyExplosion(Explosion): boolean
+ allowsBombs(): boolean
Fire
+ Fire(Vector2D, GameGrid, ImageManager):
- imageManager: ImageManager
+ getDrawing(): Drawing
+ updateInModel(): void
+ allowsBombs(): boolean
Door
+ Door(Vector2D, AbstractImage, GameGrid, Supplier<Enemy>):
+ pick(BomberMan): void
Entity
# Entity(Vector2D, GameGrid):
+ SIZE_IN_MODEL: int
- gameGrid: GameGrid
- oldDrawing: Maybe<Drawing>
+ getDrawing(AbstractImage): Drawing
+ allowsBombs(): boolean
+ hasChanged(): void
+ updateInModel(): void
+ toString(): String
+ notifyExplosion(Explosion): boolean
+ getCenterInGridCoord(): Vector2D
+ getUnderlyingFreeCell(): FreeCell
+ getDrawing(): Drawing
FreeCell
+ FreeCell(Vector2D, GameGrid):
- fires: Set<Fire>
- bomberMan: Maybe<BomberMan>
- entities: Set<Entity>
- GREEN_BACKGROUND: String
- powerup: Maybe<Powerup>
- bomb: Maybe<Bomb>
- door: Maybe<Door>
- wall: Maybe<Wall>
+ addBomb(Bomb): void
+ removePowerup(Powerup): void
+ contains(Entity): boolean
+ updateInModel(): void
+ addWall(Wall): void
+ addDoor(Door): void
+ removeFire(Fire): void
+ getEntitiesOn(): Set<Entity>
+ removeCharacter(GameCharacter): void
+ addFire(Fire): void
+ notifyExplosion(Explosion): boolean
+ addGameCharacter(GameCharacter): void
+ addBlock(Block): void
+ allowsBombs(): boolean
+ removeWall(Wall): void
+ addPowerUp(Powerup): void
+ removeBomb(Bomb): void
+ getDrawing(): Drawing
Wall
+ Wall(Vector2D, GameGrid, ImageManager, ImageManager):
- state: AbstractState
+ updateInModel(): void
+ notifyExplosion(Explosion): boolean
+ allowsBombs(): boolean
+ setState(AbstractState): void
+ getDrawing(): Drawing
<<interface>>
StateFul
+ setState(AbstractState): void
DeadState
# DeadState(StateFul):
+ notifyExplosion(Explosion): boolean
BomberManDeadState
+ BomberManDeadState(BomberMan, ImageManager):
- bomberMan: BomberMan
- deathImageManager: ImageManager
+ updateInModel(): void
SimpleDeadState
+ SimpleDeadState(T, ImageManager):
- deathImageManager: ImageManager
+ updateInModel(): void
# getEntity(): Entity
+ getDrawing(): Drawing
EnemyDeadState
+ EnemyDeadState(Enemy, ImageManager):
+ updateInModel(): void
EntityHierarchy
- EntityHierarchy():
- drawingHierarchy: Map<Class<Entity>, Integer>
- instance: EntityHierarchy
- getAssociation(Map<Class<Entity>, Integer>, Entity): Maybe<Integer>
+ getDrawLevelFor(Entity): Maybe<Integer>
GameGridBuilder
+ GameGridBuilder(Vector2D):
- grid: Maybe<GameGrid>
- topLeft: Vector2D
+ addDoor(Vector2D): void
+ getGrid(): GameGrid
+ addBomberMan(Vector2D, BomberManCapabilities): void
+ addPowerUp(Vector2D, int): void
+ addEnemy(Vector2D, int): void
+ addBlock(Vector2D): void
+ setup(Vector2D): void
+ addFreeCell(Vector2D): void
GamePlayException
+ GamePlayException(String):
WallDeadState
+ WallDeadState(Wall, ImageManager):
PlayerBuilderException
+ PlayerBuilderException(String):
Enemy
+ Enemy(Vector2D, double, boolean, GameGrid, MovingImageManager,
- previousMoveDirection: Maybe<Direction>
- streak: int
- deathPoints: int
- MEAN_STREAK: int
- stepSize: double
- getNextStreak(): int
+ move(): void
+ getDeathPoints(): int
+ getPreviousMoveDirection(): Maybe<Direction>
BomberMan
+ BomberMan(Vector2D, BomberManCapabilities, GameGrid, MovingImageM
- HORIZONTAL_MOVE_SOUND_FILE: String
- hasSteppedOnDoor: boolean
- directions: Set<Direction>
- BOMB_PLACED_SOUND: String
- bombs: Queue<Bomb>
- soundHandle: Maybe<AudioHandle>
- INVINCIBILITY_DURATION_IN_MILLIS: long
- BOMBERMAN_DEATH_SOUND: String
- stepSize: double
- invincibilityTimer: Maybe<Timer>
+ setCapabilities(GameCharacterCapabilities): void
+ setSoundHandle(Maybe<AudioHandle>): void
- update(KeyBoardUpdateInfo): void
+ stopListening(): void
+ caughtByEnemy(): void
+ update(Observable): void
+ bombHasExploded(Bomb): void
+ keyCodeToDirection(KeyBoardControllerKeyCode): Maybe<Direction>
+ move(): void
+ takePowerup(Powerup): void
- checkCoordinateAccess(Vector2D, Maybe<Direction>): boolean
+ allowsBombs(): boolean
+ updateInModel(): void
+ getSoundHandle(): Maybe<AudioHandle>
+ getCapabilities(): BomberManCapabilities
+ getBombermanDeathSound(): String
+ getDirections(): Set<Direction>
+ steppedOnDoor(Door): void
MoveManager
+ MoveManager(GameGrid):
- MODEL_SQUARE_OFFSET: int
- gameGrid: GameGrid
- generalCanAccess(Stream<Vector2D>, GameCharacterCapabilities, Vector2
+ isOnAliveWall(Vector2D): boolean
+ isOnBomb(Vector2D): boolean
+ getTopRight(Vector2D): Vector2D
+ getBottomLeft(Vector2D): Vector2D
- isOnPowerUp(Vector2D): boolean
+ checkCoordinate(Vector2D, GameCharacterCapabilities, Vector2D, Maybe<
+ isOnBlock(Vector2D): boolean
+ getBottomRight(Vector2D): Vector2D
+ isAtCrossroad(Vector2D): boolean
+ canAccessForSmallEntity(BomberManCapabilities, Vector2D, Vector2D, Dir
EntityFactory
- EntityFactory():
- UP: String
- CHARACTER_SPECIFICATION_FILE: String
- SPEED: String
- GOES_THROUGH_WALLS_KEY: String
- DEATH: String
- DOWN: String
- ALIVE: String
- entitySpecifications: JSONObject
- RIGHT: String
- factory: EntityFactory
- LEFT: String
+ getInstance(): EntityFactory
+ buildDoor(Vector2D, GameGrid): Door
+ buildPowerUp(int, Vector2D, GameGrid): Powerup
+ buildFire(FireType, Vector2D, GameGrid): Fire
- getEnemySpecification(int): JSONObject
+ buildWall(Vector2D, GameGrid): Wall
+ getEnemyCapabilities(int): GameCharacterCapabilities
- getImageListFromJsonArray(JSONArray): List<AbstractImage>
+ buildBomb(Vector2D, int, BomberMan, GameGrid, BombConstructor): Bomb
- getImagesList(JSONObject, String): List<AbstractImage>
+ buildBomberMan(Vector2D, BomberManCapabilities, GameGrid): BomberMan
+ buildEnemy(int, Vector2D, GameGrid): Enemy
PlayerBuilder
- PlayerBuilder():
- BOMB_POWER: String
- SPEED: String
- REMOTE_CONTROL: String
- GOES_THROUGH_FIRE: String
- NO_OF_LIVES: String
- instance: PlayerBuilder
- IS_INVINCIBLE: String
- GOES_THROUGH_BOMBS: String
- AVAILABLE_BOMBS: String
- LEVEL: String
- AVATAR_PATH: String
- START_LEVEL: int
- GOES_THROUGH_WALLS: String
- POINTS_EARNED: String
+ savePlayer(Player): void
- getFilePath(): Path
+ retrievePlayer(String): Maybe<Player>
+ buildPlayer(String): void
- getPlayersJson(): JSONObject
+ getDefaultAccount(String): Player
+ getInstance(): PlayerBuilder
- saveFile(JSONObject): void
GameGrid
+ GameGrid(Vector2D, Vector2D, GameGridBuilder):
- numberOfEnemies: int
- moveManager: MoveManager
- ALL_ENEMIES_ARE_DEAD_SOUND: String
- gameGridBuilder: GameGridBuilder
- topLeftCorner: Vector2D
- deadEnemiesSoundPlayer: CallbackOnce
+ update(): void
+ getCenterInGridCoord(Vector2D): Vector2D
+ doWhenTimeHasEnded(): void
+ addFire(Fire, Vector2D): void
+ addGameCharacter(GameCharacter, Vector2D): void
+ addWall(Wall, Vector2D): void
+ getCenter(Vector2D): Vector2D
+ gridCoord2modelCoor(Vector2D): Vector2D
+ addBlock(Block, Vector2D): void
+ addPowerup(Powerup, Vector2D): void
+ modelCoord2GridCoord(Vector2D): Vector2D
+ getMoveManager(): MoveManager
+ addDoor(Door, Vector2D): void
+ enemyIsDead(): void
+ moveCharacter(GameCharacter, Vector2D, Vector2D): void
+ finalize(): void
+ getTopLeftCornerModelCoordinate(): Vector2D
+ getNumberOfEnemiesStillAlive(): int
+ getDrawing(): Drawing
+ addGameCharacterInFreeCell(GameCharacter, Vector2D): void
+ addBomb(Bomb, Vector2D): void
Explosion
+ Explosion(int, Vector2D):
- fromGridCoordinate: Vector2D
- strength: int
+ getStrength(): int
BomberManCapabilities
+ BomberManCapabilities(double, int, int, boolean, boolean, bool
- goesThroughFire: boolean
- availableBombs: int
- remoteControl: boolean
- numberOfLives: int
+ defaultCapabilities: BomberManCapabilities
- isInvincible: boolean
- goesThroughBombs: boolean
- bombPower: int
- score: int
+ getAvailableBombs(): int
+ getNumberOfLives(): int
+ toString(): String
+ addPoints(int): void
+ hasRemoteControl(): boolean
+ addLife(): void
+ goesThroughFire(): boolean
+ unsetIsInvincible(): void
+ getPointsEarned(): int
+ goesThroughBombs(): boolean
+ addDelta(BomberManCapabilities): void
+ isInvincible(): boolean
+ removeLife(): void
+ setNumberOfLives(int): void
+ takesPowerUps(): boolean
+ getBombPower(): int
Model
- Model():
- dimensions: Vector2D
- game: Maybe<Game>
- controller: Controller
- instance: Model
- player: Player
- logger: Logger
- keyBoardUpdateInfo: Maybe<KeyBoardUpdateInfo>
+ close(): void
+ resetGame(): void
+ setController(Controller): void
+ getInstance(): Model
+ getGame(): Game
+ getAccount(): Player
+ getDimensions(): Vector2D
+ setAccount(Player): void
+ keyBoardEvent(KeyBoardUpdateInfo): void
+ update(): Drawing
Player
+ Player(String, int, Maybe<Path>, BomberManCapabilities):
- capabilities: BomberManCapabilities
- level: int
- playerName: String
- avatarPath: Maybe<Path>
+ setAvatar(Path): void
+ getLevel(): int
+ incrementLevel(): void
+ setLevel(int): void
+ getPlayerName(): String
+ getAvatar(): Maybe<Path>
+ getCapabilities(): BomberManCapabilities
GameGridBuilderDirector
+ GameGridBuilderDirector(GameGridBuilder, int, BomberManCapabili
- gridWidth: int
- levelNumber: int
- capabilities: BomberManCapabilities
- NUMBER_OF_POWERUPS: int
- bombermanEntryPointGridRelative: Vector2D
- gridHeight: int
- LEVEL_JSON_FILE: String
- builder: GameGridBuilder
+ allCoordinatesButBlocks(): Stream<Vector2D>
+ construct(): void
- levelAgnosticConstruction(): void
Game
+ Game():
- TIME: long
- exiting: boolean
- UPDATE_FREQ: int
- WIN_SOUND: String
- currentlyPlayingBackgroundNoiseName: String
- lastUpdate: Maybe<Pair<Long, Drawing>>
- gameGrid: GameGrid
- POWERUP_TAKEN_BACKGROUND_NOISE: String
- NORMAL_BACKGROUND_NOISE: String
- GAME_OVER_SOUND: String
- timerToEOG: Timer
- func: CallbackOnce
- handle: EnhancedAudioHandle
+ update(): Drawing
+ endGame(boolean): void
+ secsToEnd(): long
+ exited(): boolean
+ exiting(): boolean
+ getDimensions(): Vector2D
+ powerUpTaken(): void